JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JomCode Coding Bootcamp

CHAPTER 15

FastAPI with PostgreSQL and application structure

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

A database lets API data survive server restarts.

TECHNICAL EXPLANATION

Psycopg connections execute parameterized SQL. Connection contexts commit on success and roll back on failure. Routers group endpoints; services hold reusable rules.

REAL-WORLD USE

Create persistent CRUD endpoints and recognize Postlight's routers/services/database structure.

QUICK TIP

Never combine user input into SQL using f-strings. Parameters carry values separately.

INSTALL / RUN

Install: python -m pip install fastapi uvicorn python-dotenv "psycopg[binary]". Set SUPABASE_DB_URL in .env. Run schema.sql in a practice database. Start workbook:app with Uvicorn as in Chapter 14.

JOMHACK
EMPOWERING INNOVATION

Supabase database connection string

01 Supabase Dashboard > your practice project > Connect > Session pooler.

02 Copy the complete PostgreSQL URI. Preserve the supplied pooler hostname, username and port 5432.

03 Replace YOUR_PASSWORD with the database password, URL-encoding special characters such as @, # and /. This is not an API key.

04 Save the result as SUPABASE_DB_URL in .env beside workbook.py. Add sslmode=require, using & if the URI already has a query string.

Official reference

Supabase database connection string: commands

terminal / configuration

```
SUPABASE_DB_URL=postgresql://postgres.PROJECT_REF:ENCODED_PASSWORD@POOLER_HOST:5432/postgres?sslmode
=require
```

PRO TIP Run commands in the folder and operating system stated on the preceding page.

Connection check and server startup

Run schema.sql once in Supabase SQL Editor before querying the table.

Install fastapi, uvicorn, python-dotenv and psycopg[binary] in the chapter virtual environment.

connection_check.py verifies a real connection without printing credentials.

Start workbook:app, open /docs, then run GET /lesson-posts. Expected initial response: [].

```
python connection_check.py
python -m uvicorn workbook:app --reload --port 8000
```

Worked example 1: database setup

schema.sql

```sql
-- Chapter 15: run once in Supabase SQL Editor, in a practice project.
create table if not exists public.jomcode_lesson_posts (
 id bigint generated always as identity primary key,
 title text not null check (length(trim(title)) between 1 and 160)
);
alter table public.jomcode_lesson_posts enable row level security;
-- No browser policies. The lab server uses a privileged database connection.
```

PRO TIP Keep this setup once at the top of the file.

Worked example 1: shared Python setup

workbook.py

```
01    import os
02    from dotenv import load_dotenv
03    import psycopg
04    from psycopg.rows import dict_row
05    from fastapi import FastAPI, APIRouter, HTTPException
06    from pydantic import BaseModel, Field
07    load_dotenv()
08    app = FastAPI(title="Chapter 15 practice API")
09    router = APIRouter(prefix="/lesson-posts")
10
11    class PostInput(BaseModel):
12        title: str = Field(min_length=1, max_length=160)
13
14    def connection():
15        return psycopg.connect(os.environ["SUPABASE_DB_URL"])
```

PRO TIP Keep this setup once at the top of the file.

Worked example 1: Read persistent rows

workbook.py

```
@router.get("")
def list_posts():
    with connection() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute("select * from jomcode_lesson_posts order by id")
            return cur.fetchall()
```

PRO TIP This practice table is separate from the guided project's public.posts.

Run and explain example 1

EXPECTED RESULT

GET /lesson-posts returns 200 and [] before inserting data.

WHY IT BEHAVES THIS WAY

The connection and cursor close when their with blocks end. dict_row makes named fields available.

Worked example 2: Create using parameters

workbook.py

```
@router.post("", status_code=201)
def create_post(payload: PostInput):
    with connection() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                "insert into jomcode_lesson_posts(title) values (%s) returning *",
                (payload.title.strip(),))
            return cur.fetchone()
```

PRO TIP Database constraints provide a second validation layer.

Run and explain example 2

EXPECTED RESULT

POST {"title":"Hello"} returns 201 and a generated id. GET still contains it after a server restart.

WHY IT BEHAVES THIS WAY

%s is a value placeholder. A one-value parameter tuple needs its trailing comma.

Worked example 3: Update or return 404

workbook.py

```
@router.put("/{post_id}")
def update_post(post_id: int, payload: PostInput):
    with connection() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                "update jomcode_lesson_posts set title=%s where id=%s returning *",
                (payload.title.strip(), post_id))
            row = cur.fetchone()
            if row is None:
                raise HTTPException(404, "Post not found")
            return row
```

PRO TIP Keep HTTP handling in routers and shared validation in service functions.

Run and explain example 3

EXPECTED RESULT

PUT an existing id returns its updated row. An unknown id returns 404.

WHY IT BEHAVES THIS WAY

RETURNING retrieves the changed row; no matching row produces None.

Worked example 4: Delete with no response body

workbook.py

```
from fastapi import Response
@router.delete("/{post_id}", status_code=204)
def delete_post(post_id: int):
    with connection() as conn:
        with conn.cursor() as cur:
            cur.execute("delete from jomcode_lesson_posts where id=%s", (post_id,))
            if cur.rowcount == 0:
                raise HTTPException(404, "Post not found")
    return Response(status_code=204)
```

PRO TIP This practice API has no authentication. Chapter 16 adds ownership checks.

Run and explain example 4

EXPECTED RESULT

Existing id: 204 with empty body. Repeating deletion: 404.

WHY IT BEHAVES THIS WAY

The transaction commits before returning success. The client must not parse a 204 body.

Exercise 1: Single-record endpoint

YOUR TASK

Add GET /lesson-posts/{post_id} with a 404 for missing records.

CHECK THESE CASES

Existing id returns one object. Unknown id returns 404.

HINT AFTER A TRY

SELECT with a parameter and check fetchone().

Exercise 2: Reusable validator

YOUR TASK

Add a checked endpoint that trims and validates a title before inserting it.

CHECK THESE CASES

Whitespace returns 422. A valid title persists without surrounding whitespace.

HINT AFTER A TRY

Extract validation into a function.

Router registration after all endpoint definitions

workbook.py

```
01    app.include_router(router)
```

PRO TIP Keep this line after every route; it is shared application wiring, not an exercise solution.

Database connection check

connection_check.py

```
from dotenv import load_dotenv
import os
import psycopg
load_dotenv()
with psycopg.connect(os.environ["SUPABASE_DB_URL"]) as conn:
    print(conn.execute("select 1").fetchone()[0])
```

PRO TIP Expected result: 1. Diagnose connection failures before testing HTTP endpoints.

Course backend handoff to Chapter 16

01 Your Chapter 15 CRUD API is the first database checkpoint. Keep its practice table for the next chapter.

02 Chapter 16 extends that table with an author and body, and changes the teaching API from /lesson-posts to /posts.

03 Use workbooks/course_backend/checkpoints/ch15 as a recovery copy, or keep your working application.

04 The final course_backend folder is the completed answer checkpoint for Chapters 19-20. Build toward it through the listed chapter files.

Checkpoint before moving on

01 Run and explain the four worked examples, including their shared setup.

02 Attempt both exercises before reading their solutions. Check the stated acceptance cases.

03 Explain the real-world connection: Create persistent CRUD endpoints and recognize Postlight's routers/services/database structure.

Official reference